

I ran RAG on an *8GB laptop.*

No GPU. No LangChain.

One model took **40–60 minutes** to answer a single question.

6 lessons no tutorial taught me →

Chunking is a *semantic* decision, not a technical one.

policy_document.txt

— FIXED-SIZE (512 tokens)

"...reimbursement up to \$500 provided the"

x missing: who qualifies

"expense was pre-approved by a manager..."

x missing: the amount

— PARAGRAPH-BASED

"...up to \$500 provided the expense was
pre-approved by a line manager..."

✓ one complete idea

Overlap is for the *LLM*. Not for retrieval.

It barely changes similarity scores. What it does is stop chunk 7 from starting cold.

WITHOUT OVERLAP

Chunk 7 starts cold

The LLM misses the condition set in paragraph 6. The answer is incomplete — or invented.

WITH 2-SENTENCE OVERLAP

Chunk 7 has context

The last 2 lines of paragraph 6 lead into chunk 7. The answer stays grounded.

Small models don't fail loudly. They fail *confidently*.

benchmark — top-K=5 · CPU only · 8GB		
MODEL	TOK/MIN	FAILURE MODE
DeepSeek 4b	3-5	Fact blending ✗
Qwen 1.5b	7-9	Truncation △
Qwen 7b	11-12	None observed ✓

A number that *almost* matches the policy. The hardest hallucination to catch.

Citations are your chunking strategy's *unit test*.

answer — with sources

Answer: Employees are entitled to reimbursement up to \$500...



Sources:

1. policy_handbook.txt lines 42-67
2. expense_guidelines.txt lines 12-28

wrong answer? open lines 42-67,
check the chunk boundary.

ChromaDB isn't magic. It's *two systems* working as a team.

./chroma_store — on disk

```
./chroma_store/  
├─ chroma.sqlite3      ← ids, text, metadata  
└─ <collection-uuid>/  
    └─ data_level0.bin ← the vectors (HNSW)
```

HNSW does the math — finds the nearest vectors.
SQLite keeps the meaning — returns their text.

When does RAG need to become an *agent*?

01 Multi-step questions

One retrieval can't answer it honestly.

02 Multiple sources

The answer lives across documents, APIs, databases.

03 Verification required

Benefits, healthcare, compliance: check before answering.

04 Follow-up questions

The next question depends on the last answer.



RAG from *Scratch* — every failure written down.

- Prologue Building RAG from scratch
- Part 1 RAG without LangChain
- Part 2 Chunking, hallucination, token speed
- Part 3 The benchmark experiments
- Part 4 From RAG to agentic RAG
- Part 5 ChromaDB → S3 Vectors

yesprasad.com/series/rag-from-scratch